20.1 WhatsApp 通道（Baileys）

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~600k tokens，输出 ~55k tokens（本章合计）

WhatsApp 是全球使用最广泛的即时通信应用之一。OpenClaw 通过 **Baileys** 库实现了 WhatsApp 通道——这是一个基于 WhatsApp Web 协议的非官方 Node.js 实现。本节将深入分析从 QR 码登录到消息收发的完整流程。

20.1.1 Baileys 库简介

> **衍生解释：Baileys 与 WhatsApp Web 协议**
>
> WhatsApp 官方并没有提供公开的 Bot API（与 Telegram 不同）。Baileys（`@whiskeysockets/baileys`）是社区维护的开源库，通过逆向工程 WhatsApp Web 的 WebSocket 协议来实现程序化的消息收发。它模拟了浏览器端的 WhatsApp Web 客户端，使用"关联设备"（Linked Devices）机制连接到 WhatsApp 服务器。
>
> 这意味着：
>
> - 使用的是**真实的 WhatsApp 号码**，而非专用的 Bot 账号
> - 连接方式是扫描 QR 码进行**设备配对**
> - 受限于 WhatsApp 的使用条款，存在被封号的风险
> - 建议使用独立的 SIM 卡/eSIM 号码

OpenClaw 的 WhatsApp 模块主要位于 `src/web/` 目录下（历史原因，"web" 指 WhatsApp Web）。

20.1.2 WhatsApp Web 登录与 QR 码配对

Socket 创建

登录流程的核心是 `createWaSocket` 函数（`src/web/session.ts`），它封装了 Baileys 的 `makeWASocket`：

```ts
// src/web/session.ts（简化）
export async function createWaSocket(
  printQr: boolean,
  verbose: boolean,
  opts: { authDir?: string; onQr?: (qr: string) => void } = {},
) {
  const { state, saveCreds } = await useMultiFileAuthState(authDir);
  const { version } = await fetchLatestBaileysVersion();

  const sock = makeWASocket({
    version,
    auth: { creds: state.creds, keys: makeCacheableSignalKeyStore(state.keys, logger) }
    printQRInTerminal: printQr,
    // ...
  });

  // 凭证变更时自动保存
  sock.ev.on("creds.update", () => {
    enqueueSaveCreds(authDir, saveCreds, logger);
  });

  // QR 码回调
  sock.ev.on("connection.update", (update) => {
    if (update.qr && opts.onQr) {
      opts.onQr(update.qr);
    }
  });

  return sock;
}
```

Baileys 使用 **Signal 协议** 的密钥存储机制（ `useMultiFileAuthState` ）将凭证持久化到磁盘。 `makeCacheableSignalKeyStore` 对密钥操作进行缓存，减少文件系统 I/O。

> **衍生解释：Signal 协议**
>
> Signal 协议是一种端到端加密通信协议，由 Signal 基金会开发。WhatsApp 采用了 Signal 协议来加密用户消息。Baileys 需要维护一组 Signal 协议密钥（身份密钥、预密钥、会话密钥等），这些密钥存储在 `authDir` 目录下。

QR 码登录流程

`startWebLoginWithQr` （`src/web/login-qr.ts`）实现了完整的 QR 码登录：

```
// src/web/login-qr.ts（简化）
export async function startWebLoginWithQr(opts): Promise<{ qrDataUrl?; message }> {
  const account = resolveWhatsAppAccount({ cfg, accountId });

  // 已登录则提示
  if (await webAuthExists(account.authDir) && !opts.force) {
    return { message: `WhatsApp is already linked (${who}).` };
  }

  // 复用未过期的活跃登录
  const existing = activeLogins.get(account.accountId);
  if (existing && isLoginFresh(existing) && existing.qrDataUrl) {
    return { qrDataUrl: existing.qrDataUrl, message: "QR already active." };
  }

  // 创建新 Socket 并等待 QR 码
  const sock = await createWaSocket(false, verbose, {
    authDir: account.authDir,
    onQr: (qr) => { resolveQr(qr); },
  });

  // 将 QR 渲染为 PNG 的 base64 Data URL
  const qr = await qrPromise;
  const base64 = await renderQrPngBase64(qr);
  login.qrDataUrl = `${base64}`;

  return { qrDataUrl: login.qrDataUrl, message: "Scan this QR in WhatsApp → Linked Devi
}
```

活跃登录有 3 分钟的 TTL（`ACTIVE_LOGIN_TTL_MS = 3 * 60_000`），过期后需要重新生成 QR 码。

连接等待与错误恢复

`waitForWebLogin` 等待扫码完成，并处理各种错误：

```
// src/web/login-qr.ts（简化）
export async function waitForWebLogin(opts): Promise<{ connected; message }> {
  while (true) {
    await Promise.race([login.waitPromise, timeout]);

    if (login.error) {
      // 被 WhatsApp 登出——清除缓存凭证
      if (login.errorStatus === DisconnectReason.loggedOut) {
        await logoutWeb({ authDir: login.authDir });
        return { connected: false, message: "Session logged out. Please scan a new QR."
      }
      // 515 错误——WhatsApp 要求重连
      if (login.errorStatus === 515) {
        const restarted = await restartLoginSocket(login, runtime);
        if (restarted) continue;   // 重新等待
      }
      return { connected: false, message: `Login failed: ${login.error}` };
    }

    if (login.connected) {
      return { connected: true, message: "✅ Linked! WhatsApp is ready." };
    }
  }
}
```

515 错误处理值得单独说一下：WhatsApp 服务器在配对后有时会要求客户端重启连接（状态码 515）。OpenClaw 对此进行了**一次性自动重连**（`restartAttempted` 标志防止无限重试）。

20.1.3 入站消息监听

入站消息的监控通过 `monitorWebInbox`（`src/web/inbound/monitor.ts`）实现，它订阅 Baileys Socket 的消息事件，将 WhatsApp 消息转换为 `WebInboundMessage` 结构，再流入统一的 `MsgContext` 管线。

关键的提取函数包括：

- `extractText`：从 WhatsApp 消息对象中提取纯文本（处理 conversation、extendedTextMessage、imageMessage.caption 等多种格式）
- `extractMediaPlaceholder`：提取媒体占位符信息
- `extractLocationData`：提取位置信息

入站消息经过白名单过滤（`allowFrom` 列表）后才会进入处理管线。

20.1.4 出站消息发送

`sendMessageWhatsApp`（`src/web/outbound.ts`）负责将 AI 回复发送到 WhatsApp：

```
// src/web/outbound.ts（简化）
export async function sendMessageWhatsApp(to, body, options) {
  let text = body;
  const { listener: active } = requireActiveWebListener(options.accountId);

  // Markdown 表格转换
  const tableMode = resolveMarkdownTableMode({ cfg, channel: "whatsapp" });
  text = convertMarkdownTables(text, tableMode);

  const jid = toWhatsappJid(to);

  // 媒体处理
  if (options.mediaUrl) {
    const media = await loadWebMedia(options.mediaUrl);
    // ...根据媒体类型设置发送参数
  }

  // 先发送"正在输入"状态
  await active.sendComposingTo(to);

  // 发送消息
  const result = await active.sendMessage(to, text, mediaBuffer, mediaType);
  return { messageId, toJid: jid };
}
```

此外还支持 `sendReactionWhatsApp`（表情反应）和 `sendPollWhatsApp`（投票）。WhatsApp 的投票最多支持 12 个选项。

20.1.5 自动回复系统

WhatsApp 的自动回复核心在 `src/web/auto-reply/monitor.ts`（`monitorWebChannel`），它将入站消息监听与 AI Agent 调度连接起来，形成完整的"收消息 → AI 处理 → 发回复"闭环。

20.1.6 会话重连

`src/web/reconnect.ts` 定义了断线重连策略：

```
// src/web/reconnect.ts
export const DEFAULT_RECONNECT_POLICY: ReconnectPolicy = {
  initialMs: 2_000,      // 首次重连延迟 2 秒
  maxMs: 30_000,         // 最大延迟 30 秒
  factor: 1.8,           // 指数退避因子
  jitter: 0.25,          // 25% 随机抖动
  maxAttempts: 12,        // 最多重试 12 次
};
```

> **衍生解释：指数退避（Exponential Backoff）**
>
> 指数退避是一种重试策略，每次失败后等待时间呈指数增长：第 1 次等 2s，第 2 次等 3.6s（2×1.8），第 3 次等 6.48s...直到达到最大延迟。加入随机抖动（jitter）可以防止多个客户端在同一时刻发起重连（"惊群效应"）。

20.1.7 媒体处理

`loadWebMedia`（`src/web/media.ts`）处理媒体文件的加载和优化：

```
// src/web/media.ts（简化）
export async function loadWebMedia(mediaUrl, maxBytes?): Promise<WebMediaResult> {
  // 支持 file:// URL、HTTP URL 和本地路径
  // MIME 类型通过魔数嗅探检测
  // 图片自动优化（HEIC→JPEG、尺寸缩放、质量调整）
  // 尺寸/质量网格搜索：sides=[2048,1536,1280,1024,800] × qualities=[80,70,60,50,40]
}
```

图片优化使用网格搜索策略——尝试不同的尺寸和质量组合，找到第一个满足大小限制的组合。还支持 **HEIC → JPEG** 转换（iPhone 拍摄的照片通常是 HEIC 格式），以及透明 PNG 的特殊处理（保留 alpha 通道）。

本节小结

1. **Baileys** 是基于 WhatsApp Web 协议的非官方库，使用真实号码通过 QR 码配对登录。
2. **QR 码登录**有 3 分钟 TTL，支持 515 错误自动重连和凭证安全备份。
3. **入站消息**经过格式提取和白名单过滤后进入统一处理管线。
4. **出站消息**自动进行 Markdown 表格转换，支持文本、媒体、反应和投票。
5. **断线重连**使用带抖动的指数退避策略，最多重试 12 次。
6. **媒体处理**支持自动图片优化、HEIC 转 JPEG、MIME 嗅探等，通过网格搜索找到满足大小限制的最优质量。